AMRITA VISHWA VIDYAPEETHAM

Course Code: 19ECE347

Course Name: Electronic System Level Design and Verification

Component: Simulation and Analysis-based Assignment

Date of Evaluation: 4th November 2025

Academic Year: 2025-2026 (Odd Semester)

Batch Number : 4

Name of the Students With Registration no.

1) Ayush Singh
BL.EN.U4ECE22210

2) Gokul Hari Shankar
BL.EN.U4ECE22217

3) Muzammil Pasha
BL.EN.U4ECE22230

Branch: ECE

Semester: VII

Batch: 2022-2026

Faculty In-charge: Mrs. Sonali Agrawal

Aim:
To design and verify simplified AMBA AHB system

Tools Used:
ModelSim, Xilinx Vivado

Theory:

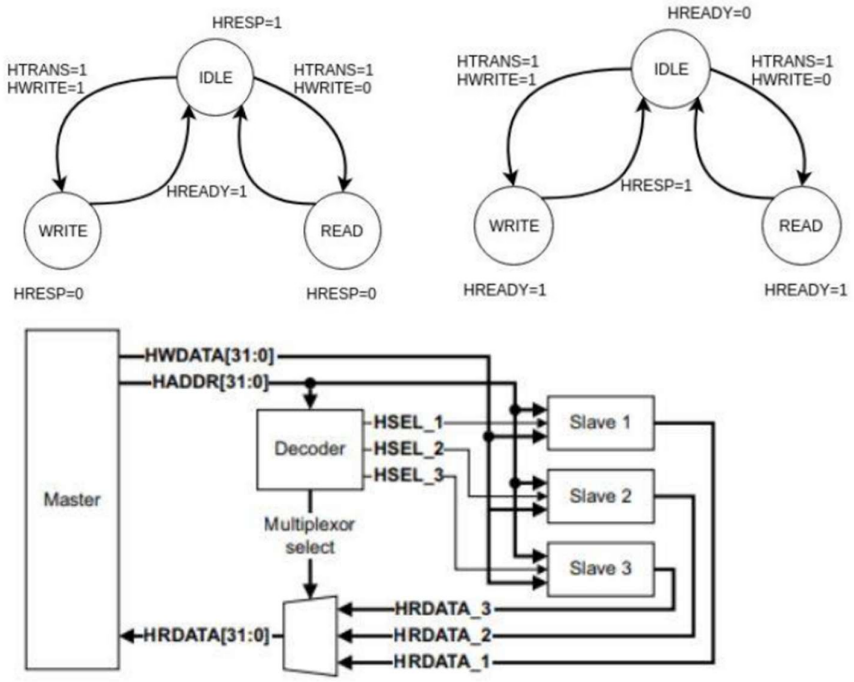
1) Introduction

   AMBA (Advanced Microcontroller Bus Architecture) is a communication standard developed by ARM to connect different parts of a System-on-Chip (SoC), like CPU, memory, and input/output devices. One of the most important buses in AMBA is the **AHB – Advanced High-performance Bus**.

   The **AMBA AHB** is used when fast data transfer is required. It connects high-speed components such as processors, on-chip memory, and DMA controllers. In this bus system, only one master can use the bus at a time to communicate with a slave device. The communication is controlled using signals like address, data, and control signals.

   The AHB bus uses features like **pipelining**, **burst** and **single clock operation**, which help in transferring data quickly and efficiently. Because AMBA AHB follows a standard format, it becomes easier to design systems and reuse the same blocks in other projects.

2) Logic Circuit or Block Diagram



3) Working principle

In the AMBA AHB system, communication happens between **one master and 3 slaves** using a **shared bus**, an **address decoder**, and a **multiplexer**. Both the master and the slave are designed using **finite state machines (FSMs)** with three states: **IDLE, READ, and WRITE**.

Master Side Operation

The master remains in the **IDLE** state until it needs to perform a transfer.

- If it wants to **write**, it sets HTRANS = 1 and HWRITE = 1.
- If it wants to **read**, it sets HTRANS = 1 and HWRITE = 0.

Based on this, the master FSM moves to the **READ** or **WRITE** state. It places the **address on HADDR**, and in case of write, **data on HWDATA**. These address and data signals are broadcast to all slaves.

Slave Selection using Decoder (Figure 5)

The address (HADDR) is sent to the **Decoder**, which checks the **last two bits of the address**. These two bits decide which slave must be activated.

- Depending on the address, the decoder enables a slave by asserting its **HSELx** signal.
- Only the selected slave will respond; others stay inactive.

Slave Operation (Figure 4 & FSM in Figure 6)

Once HSELx = 1, the slave becomes active and moves out of the **IDLE** state.

- If it is a **read** (HTRANS = 1, HWRITE = 0), the slave reads data from the specified address and places it on HRDATA.
- If it is a **write** (HTRANS = 1, HWRITE = 1), it takes data from HWDATA and stores it at the address received on HADDR.

After completing the operation, the slave sets **HREADY = 1** to indicate that data is valid (for read) or write is completed.
The master checks HREADY; once it is high, it considers the transfer complete and returns to IDLE.

Data Return to Master using Multiplexer (Figure 5)

All slaves send their output data (HRDATA_x) to the **slave-to-master multiplexer**.
Only the selected slave's data is passed to the master through this multiplexer.
This ensures that the master only receives the correct data from the active slave.

Completion of Transfer

Once the master receives **HREADY = 1**, it may set **HRESP = 1** to signal the end of the transaction.
After this:

- The slave returns to the **IDLE** state.
- The master either starts another transfer or stays in IDLE.

4) Advantages

Feature	Advantage
High Performance	Supports pipelined operations which reduce wait cycles and improve data throughput.
Single Clock Edge Operation	Simplifies timing analysis and synchronization across components.
Multiple Master Support	Allows more than one master (CPU, DMA, etc.), enabling efficient resource sharing with arbitration logic.
Standardized Interface	Promotes design reuse and reduces complexity when integrating IP cores from different vendors.
Low Latency Transfers	Single-cycle access for sequential data transfers ensures low latency.
Simple Architecture Compared to AXI	Fewer channels and signals than AXI, making it easier to design and implement.
Error Handling Support	Includes response signals (HRESP) to indicate OKAY, ERROR, or RETRY, improving reliability.

5) Disadvantages

Limitation	Disadvantage
No Separate Read and Write Channels	Unlike AXI, AHB cannot perform simultaneous read/write operations, limiting bandwidth.
In-Order Execution Only	Does not support out-of-order transactions or multiple outstanding transfers.
Scalability Limitation	Becomes less efficient in complex multi-layer, multi-master systems compared to AXI or AHB BusMatrix.
Higher Power Consumption vs APB	Since AHB operates at higher clock frequencies and uses more control signals, it consumes more power than APB.
Less Flexible than AXI	No s;upport for QoS (Quality of Service), cache signals, or advanced features like AXI's VALID–READY handshake.

Limitation	Disadvantage
Shared Bus Bottleneck	All masters share a single bus, which can become a bottleneck in high-traffic systems.

SystemVerilog Code:

```
`timescale 1ns/1ps
module master_fsm(
   output logic [31:0] HADDR,
   output logic        HWRITE,
   output logic [1:0]  HTRANS,
   output logic [31:0] HWDATA,
   input  logic        HRESP,
   input  logic        HRESETn,
   input  logic        HCLK,
   input  logic        HREADY,
   input  logic [31:0] HRDATA
);
   typedef enum logic [1:0] {
      IDLE  = 2'b00,
      WRITE = 2'b01,
      READ  = 2'b10,
      WAIT  = 2'b11
   } state_t;

   state_t ps, ns;

   always_ff @(posedge HCLK or negedge HRESETn) begin
      if (!HRESETn) begin
         ps     <= IDLE;
         HADDR  <= 32'h0;
         HWRITE <= 1'b0;
         HWDATA <= 32'h0;
```

```
        HTRANS <= 2'b00;
      end else begin
        ps <= ns;
        case (ns)
          IDLE:  begin HTRANS <= 2'b00; HWRITE <= 0; end
          WRITE: begin HADDR <= 32'h00000004; HWRITE <= 1; HWDATA <= 32'hA5A55A5A;
HTRANS <= 2'b10; end
          READ : begin HADDR <= 32'h00000004; HWRITE <= 0; HTRANS <= 2'b10; end
          WAIT : begin HTRANS <= 2'b00; end
        endcase
      end
    end

    always_comb begin
      ns = ps;
      case (ps)
        IDLE  : ns = WRITE;
        WRITE : if (HREADY) ns = WAIT;
        WAIT  : ns = READ;
        READ  : if (HREADY) ns = IDLE;
      endcase
    end
endmodule

module ahb_slave_fsm (
    input  logic       HRESETn,
    input  logic       HCLK,
    input  logic       HSELx,
    input  logic [31:0] HADDR,
    input  logic       HWRITE,
    input  logic [1:0]  HTRANS,
    input  logic [31:0] HWDATA,
    output logic       HREADY,
    output logic [31:0] HRDATA,
    output logic       HRESP
```

```
);

    typedef enum logic [1:0] { IDLE=2'b00, WRITE=2'b01, READ=2'b10 } state_t;
    state_t ps, ns;

    logic [31:0] mem [0:3];
    logic [31:0] addr_phase;
    logic        write_phase;
    logic        trans_active;

    always_comb begin
        ns = ps;
        case (ps)
            IDLE: begin
                if (trans_active && write_phase) ns = WRITE;
                else if (trans_active && !write_phase) ns = READ;
            end
            WRITE, READ: ns = IDLE;
        endcase
    end

    always_ff @(posedge HCLK or negedge HRESETn) begin
        if (!HRESETn) begin
            ps <= IDLE;
            HREADY <= 1;
            HRESP  <= 0;
            HRDATA <= 32'h0;
            addr_phase <= 32'h0;
            write_phase <= 0;
            trans_active <= 0;
            for (int i = 0; i < 4; i++) mem[i] <= 32'h0;
        end else begin
            if (HSELx && HTRANS == 2'b10 && HREADY) begin
                addr_phase <= HADDR;
```

```
            write_phase <= HWRITE;
            trans_active <= 1'b1;
        end else begin
            trans_active <= 1'b0;
        end

        ps <= ns;

        case (ps)
            WRITE: begin
                mem[addr_phase[3:2]] <= HWDATA;
                HREADY <= 1;
                HRESP <= 0;
                $display("[%0t] SLV WRITE: Addr=%h Data=%h", $time, addr_phase, HWDATA);
            end
            READ: begin
                HRDATA <= mem[addr_phase[3:2]];
                HREADY <= 1;
                HRESP <= 0;
                $display("[%0t] SLV READ : Addr=%h Data=%h", $time, addr_phase, mem[addr_phase[3:2]]);
            end
            default: begin
                HREADY <= 1;
                HRESP <= 0;
            end
        endcase
      end
   end
endmodule


module ahb_decoder (
   input  logic [31:0] HADDR,
   output logic [2:0]  HSEL
```

```
);
    always_comb begin
        HSEL = 3'b000;
        casez (HADDR[15:12])
            4'h0: HSEL = 3'b001; // Slave 1
            4'h1: HSEL = 3'b010; // Slave 2
            4'h2: HSEL = 3'b100; // Slave 3
            default: HSEL = 3'b000;
        endcase
    end
endmodule

module ahb_mux (
    input  logic [2:0]   HSEL,
    input  logic [31:0]  HRDATA1,
    input  logic [31:0]  HRDATA2,
    input  logic [31:0]  HRDATA3,
    input  logic         HREADY1,
    input  logic         HREADY2,
    input  logic         HREADY3,
    output logic [31:0]  HRDATA,
    output logic         HREADY
);
    always_comb begin
        case (HSEL)
            3'b001: begin HRDATA = HRDATA1; HREADY = HREADY1; end
            3'b010: begin HRDATA = HRDATA2; HREADY = HREADY2; end
            3'b100: begin HRDATA = HRDATA3; HREADY = HREADY3; end
            default: begin HRDATA = 32'h00000000; HREADY = 1'b1; end
        endcase
    end
endmodule

module ahb_top (
```

```
    input  logic        HCLK,
    input  logic        HRESETn,
    input  logic [31:0] HADDR,
    input  logic [31:0] HWDATA,
    input  logic [1:0]  HTRANS,
    input  logic        HWRITE,
    output logic [31:0] HRDATA,
    output logic        HREADY,
    output logic        HRESP
);
    logic [2:0]  HSEL;
    logic [31:0] HRDATA1, HRDATA2, HRDATA3;
    logic        HREADY1, HREADY2, HREADY3;

    assign HRESP = 1'b0;

    ahb_decoder dec (.HADDR(HADDR), .HSEL(HSEL));

    ahb_slave_fsm s1 (.HRESETn(HRESETn), .HCLK(HCLK), .HSELx(HSEL[0]),
                .HADDR(HADDR), .HWRITE(HWRITE), .HTRANS(HTRANS),
                .HWDATA(HWDATA), .HREADY(HREADY1), .HRDATA(HRDATA1), .HRESP());

    ahb_slave_fsm s2 (.HRESETn(HRESETn), .HCLK(HCLK), .HSELx(HSEL[1]),
                .HADDR(HADDR), .HWRITE(HWRITE), .HTRANS(HTRANS),
                .HWDATA(HWDATA), .HREADY(HREADY2), .HRDATA(HRDATA2), .HRESP());

    ahb_slave_fsm s3 (.HRESETn(HRESETn), .HCLK(HCLK), .HSELx(HSEL[2]),
                .HADDR(HADDR), .HWRITE(HWRITE), .HTRANS(HTRANS),
                .HWDATA(HWDATA), .HREADY(HREADY3), .HRDATA(HRDATA3), .HRESP());

    ahb_mux mux (.HSEL(HSEL),
            .HRDATA1(HRDATA1), .HRDATA2(HRDATA2), .HRDATA3(HRDATA3),
            .HREADY1(HREADY1), .HREADY2(HREADY2), .HREADY3(HREADY3),
            .HRDATA(HRDATA), .HREADY(HREADY));
```

```
endmodule


interface inter(input bit HCLK);
    logic       HRESETn;
    logic [31:0] HADDR;
    logic [31:0] HWDATA;
    logic [1:0]  HTRANS;
    logic       HWRITE;
    logic [31:0] HRDATA;
    logic       HREADY;
    logic       HRESP;
endinterface

class ahb_transaction;
    bit [31:0] addr;
    bit [31:0] wdata;
    bit [31:0] rdata;
    bit [1:0]  trans;
    bit        write;
endclass

class generator;
    mailbox gen2drv;
    event drv_done;

    function new(mailbox m); gen2drv = m; endfunction

    task run();
        ahb_transaction t;
        bit [31:0] addr_list [3] = '{32'h00000004, 32'h10000004, 32'h20000004};
        bit [31:0] data_list [3] = '{32'h11111111, 32'h22222222, 32'h33333333};

        #40;
        for (int i=0; i<3; i++) begin
```

```
            t = new(); t.addr=addr_list[i]; t.wdata=data_list[i]; t.write=1; t.trans=2'b10;
            gen2drv.put(t);
            $display("[GEN] WRITE Addr=%h Data=%h", t.addr, t.wdata);
            @(drv_done);
        end

        #20;
        for (int i=0; i<3; i++) begin
            t = new(); t.addr=addr_list[i]; t.write=0; t.trans=2'b10;
            gen2drv.put(t);
            $display("[GEN] READ  Addr=%h", t.addr);
            @(drv_done);
        end
    endtask
endclass

class driver;
    mailbox gen2drv;
    virtual inter vif;
    event drv_done;

    function new(mailbox m, virtual inter v);
        gen2drv = m; vif = v;
    endfunction

    task run();
        ahb_transaction t;
        vif.HADDR <= 0; vif.HWDATA <= 0; vif.HTRANS <= 0; vif.HWRITE <= 0;
        forever begin
            gen2drv.get(t);
            @(posedge vif.HCLK);
            vif.HADDR <= t.addr;
            vif.HWRITE <= t.write;
            vif.HTRANS <= 2'b10;
```

```
        @(posedge vif.HCLK);
        if (t.write) vif.HWDATA <= t.wdata;
        while (vif.HREADY !== 1) @(posedge vif.HCLK);
        if (!t.write)
          $display("[DRV] READ  Addr=%h Data=%h", t.addr, vif.HRDATA);
        else
          $display("[DRV] WRITE Addr=%h Data=%h", t.addr, t.wdata);
        @(posedge vif.HCLK);
        vif.HTRANS <= 0; vif.HWRITE <= 0; vif.HWDATA <= 0;
        -> drv_done;
      end
    endtask
endclass

class monitor;
    mailbox mon2scb;
    virtual inter vif;
    bit [31:0] addr_phase;
    bit write_phase;
    bit trans_pending;

    function new(mailbox m, virtual inter v);
      mon2scb=m; vif=v; trans_pending=0;
    endfunction

    task run();
      ahb_transaction t;
      forever begin
        @(posedge vif.HCLK);
        if (trans_pending && vif.HREADY) begin
          t = new();
          t.addr = addr_phase;
          t.write = write_phase;
          if (write_phase) begin
```

```
            t.wdata = vif.HWDATA;
            $display("[MON] Captured WRITE Addr=%h Data=%h", t.addr, t.wdata);
          end else begin
            t.rdata = vif.HRDATA;
            $display("[MON] Captured READ  Addr=%h Data=%h", t.addr, t.rdata);
          end
          mon2scb.put(t);
          trans_pending = 0;
        end
        if (vif.HTRANS == 2'b10) begin
          addr_phase = vif.HADDR;
          write_phase = vif.HWRITE;
          trans_pending = 1;
        end
      end
    endtask
endclass

class scoreboard;
    mailbox mon2scb;
    bit [31:0] exp_mem [bit[31:0]];

    function new(mailbox m); mon2scb = m; endfunction

    task run();
      ahb_transaction t;
      forever begin
        mon2scb.get(t);
        if (t.write) begin
          exp_mem[t.addr] = t.wdata;
          $display("[SCB] Stored WRITE addr=%h data=%h", t.addr, t.wdata);
        end else if (exp_mem.exists(t.addr)) begin
          if (exp_mem[t.addr] !== t.rdata)
            $error("[SCB] MISMATCH addr=%h exp=%h got=%h", t.addr, exp_mem[t.addr], t.rdata);
```

```
            else
                $display("[SCB] PASS addr=%h data=%h", t.addr, t.rdata);
        end
    end
  endtask
endclass

class environment;
  generator  gen;
  driver     drv;
  monitor    mon;
  scoreboard scb;
  mailbox gen2drv, mon2scb;
  virtual inter vif;

  function new(virtual inter v);
    vif = v;
    gen2drv = new();
    mon2scb = new();
    gen = new(gen2drv);
    drv = new(gen2drv, vif);
    mon = new(mon2scb, vif);
    scb = new(mon2scb);
    gen.drv_done = drv.drv_done;
  endfunction

  task run();
    fork
      gen.run();
      drv.run();
      mon.run();
      scb.run();
    join_none
  endtask
```

```
endclass

module tb_ahb;
    bit HCLK;
    always #5 HCLK = ~HCLK;

    inter vif(HCLK);
    ahb_top dut(
        .HCLK(HCLK),
        .HRESETn(vif.HRESETn),
        .HADDR(vif.HADDR),
        .HWDATA(vif.HWDATA),
        .HTRANS(vif.HTRANS),
        .HWRITE(vif.HWRITE),
        .HRDATA(vif.HRDATA),
        .HREADY(vif.HREADY),
        .HRESP(vif.HRESP)
    );

    environment env;

    initial begin
        HCLK = 0;
        vif.HRESETn = 0;
        vif.HADDR = 0;
        vif.HWDATA = 0;
        vif.HTRANS = 0;
        vif.HWRITE = 0;
        #20 vif.HRESETn = 1;
        env = new(vif);
        env.run();
        #1000 $finish;
    end
endmodule
```

Simulation Result:

```
# run 1000ns
[GEN] WRITE Addr=00000004 Data=11111111
[DRV] WRITE Addr=00000004 Data=11111111
[MON] Captured WRITE Addr=00000004 Data=11111111
[GEN] WRITE Addr=10000004 Data=22222222
[SCB] Stored WRITE addr=00000004 data=11111111
[MON] Captured WRITE Addr=00000004 Data=00000000
[SCB] Stored WRITE addr=00000004 data=00000000
[DRV] WRITE Addr=10000004 Data=22222222
[MON] Captured WRITE Addr=10000004 Data=22222222
[GEN] WRITE Addr=20000004 Data=33333333
[SCB] Stored WRITE addr=10000004 data=22222222
[MON] Captured WRITE Addr=10000004 Data=00000000
[SCB] Stored WRITE addr=10000004 data=00000000
[DRV] WRITE Addr=20000004 Data=33333333
[MON] Captured WRITE Addr=20000004 Data=33333333
[SCB] Stored WRITE addr=20000004 data=33333333
[MON] Captured WRITE Addr=20000004 Data=00000000
[SCB] Stored WRITE addr=20000004 data=00000000
[GEN] READ  Addr=00000004
[DRV] READ  Addr=00000004 Data=00000000
[MON] Captured READ  Addr=00000004 Data=00000000
[SCB] PASS addr=00000004 data=00000000
[GEN] READ  Addr=10000004
[MON] Captured READ  Addr=00000004 Data=00000000
[SCB] PASS addr=00000004 data=00000000
[DRV] READ  Addr=10000004 Data=00000000
[MON] Captured READ  Addr=10000004 Data=00000000
[GEN] READ  Addr=20000004
[SCB] PASS addr=10000004 data=00000000
[MON] Captured READ  Addr=10000004 Data=00000000
[SCB] PASS addr=10000004 data=00000000
[DRV] READ  Addr=20000004 Data=00000000
[MON] Captured READ  Addr=20000004 Data=00000000
[SCB] PASS addr=20000004 data=00000000
[MON] Captured READ  Addr=20000004 Data=00000000
[SCB] PASS addr=20000004 data=00000000
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_ahb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
```

Synthesis Report :

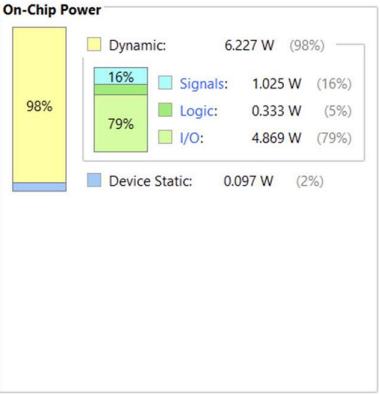
Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

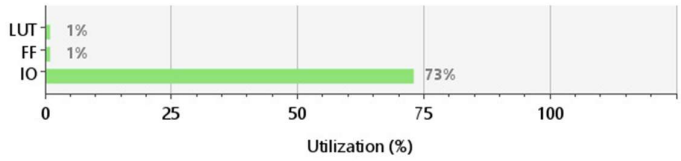
Total On-Chip Power:	**6.324 W**
Design Power Budget:	**Not Specified**
Process:	typical
Power Budget Margin:	**N/A**
Junction Temperature:	**56.6°C**
Thermal Margin:	28.4°C (5.6 W)
Ambient Temperature:	25.0 °C
Effective ΘJA:	5.0°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Low

Launch Power Constraint Advisor to find and fix invalid switching activity

On-Chip Power



Resource	Utilization	Available	Utilization %
LUT	159	20800	0.76
FF	489	41600	1.18
IO	77	106	72.64



Conclusion:

Successfully Implemented and verified simplified AMBA AHB system using 1 master block and 3 slave blocks.

Reference:

[1] P. Giridhar and P. Choudhury, "Design and Verification of AMBA AHB," 2019 1st International Conference on Advanced Technologies in Intelligent Control, Environment, Computing & Communication Engineering (ICATIECE), Bangalore, India, 2019, pp. 310-315, doi: 10.1109/ICATIECE45860.2019.9063856. keywords: {Decoding;Microcontrollers;Multiplexing;Automata;Standards;System-on-chip;Protocols;AHB;AMBA-AHB;QuestaSim;ARM}

[2] L. Deeksha and B. R. Shivakumar, "Effective Design and Implementation of AMBA AHB Bus Protocol using Verilog," 2019 International Conference on Intelligent Sustainable Systems (ICISS), Palladam, India, 2019, pp. 1-5, doi: 10.1109/ISS1.2019.8907975. keywords: {Microcontrollers;Hardware design languages;System-on-chip;Clocks;Registers;Conferences;Computer architecture;Advanced Microcontroller Bus Architecture;Advanced High-performance Bus;System-on-Chip;Hardware Description Language}